

TutorIA: Agente académico con LLM para estudiantes de Ingeniería en Sistemas

Grupo 16

Rivero Tobías Nicolás - tobirivero789@gmail.com

Gómez Enrico Ivo - ivogomezenrico2015@gmail.com

Werlen Alexander - alexwerlenuni@gmail.com

Resumen: TutorIA es un agente conversacional basado en modelos de lenguaje orientado a estudiantes de Ingeniería en Sistemas de Información. El sistema asiste al estudiante en su proceso de aprendizaje explicando conceptos, respondiendo dudas sobre el material de la materia y ayudándolo a organizar fechas de parciales, entregas y exposiciones. La solución se implementa como una aplicación web, integrada a un modelo de lenguaje de chat mediante la API de OpenCode y a un servicio de embeddings de Google para la búsqueda semántica sobre los apuntes que el estudiante sube a la plataforma. El agente opera con un esquema de function-calling nativo y dispone de herramientas para listar y leer documentos, buscar en los apuntes por similitud, consultar la agenda académica y mantener un perfil persistente del estudiante. La evaluación se realizó sobre una batería de casos que cubre escenarios principales, casos límite y entradas adversariales; los resultados muestran que el agente responde de forma consistente, cita las fuentes que utiliza y maneja correctamente los errores de herramientas y los pedidos fuera de dominio.

1 - INTRODUCCIÓN

La formación en carreras de ingeniería se caracteriza por un volumen elevado de material bibliográfico, apuntes de clase y trabajos prácticos que los estudiantes deben organizar y comprender por su cuenta. En la carrera de Ingeniería en Sistemas de Información, los apuntes suelen estar dispersos entre PDFs, archivos de texto y carpetas compartidas, y no siempre se cuenta con un canal inmediato para evacuar dudas conceptuales. Esta situación se ve agravada por la falta de herramientas que permitan al estudiante consultar en lenguaje natural sobre su propio material y, al mismo tiempo, organizar su agenda académica.

TutorIA es una propuesta de agente académico basado en modelos de lenguaje que aborda este problema. El sistema permite al estudiante mantener chats por materia y por unidad temática, subir sus apuntes y documentos, y conversar con un agente que conoce la materia que está cursando, el año de la carrera y los eventos académicos registrados. A lo largo del trabajo se describen las decisiones de diseño adoptadas, la arquitectura del sistema, el diseño del agente, la implementación de la interfaz y los resultados de la evaluación realizada.

El resto del informe se organiza de la siguiente manera. En la sección 2 se caracteriza al sistema como agente inteligente según la taxonomía clásica. En la sección 3 se describe la arquitectura general y el flujo de un request. En la sección 4 se detalla el diseño del agente: prompt, herramientas, loop, memoria, RAG, guardrails y observabilidad. En la sección 5 se comenta la implementación y la interfaz de usuario. En la sección 6 se reportan los resultados de la evaluación. Finalmente, en la sección 7 se presentan las conclusiones y el trabajo futuro.

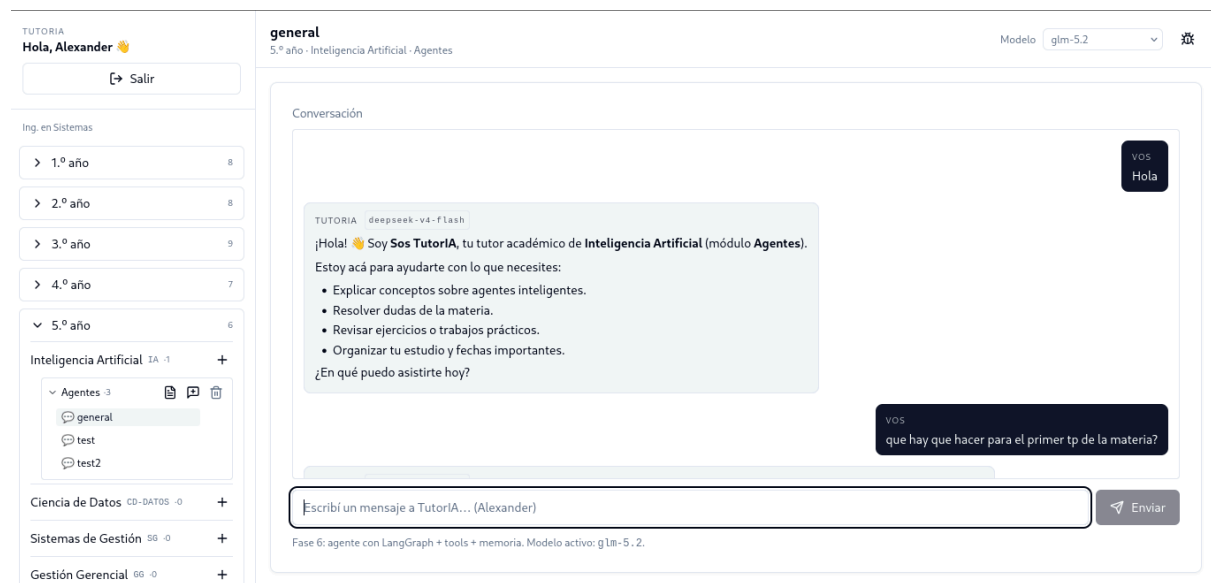


Figura 1. Vista general de TutorIA: el estudiante conversa con el agente sobre su materia y sus apuntes.

2 - Marco teórico y conceptos de agentes aplicados

TutorIA se modela como un agente racional cuyo objetivo es asistir al estudiante en su proceso de aprendizaje. El entorno del agente es la sesión de chat de una materia y un módulo específicos, junto con los documentos, los eventos académicos y las preferencias del usuario que el propio sistema mantiene. Las percepciones son los mensajes del estudiante, los resultados de las herramientas y el contexto cargado en la conversación; las acciones son las respuestas textuales y las invocaciones a las herramientas disponibles.

El entorno es parcialmente observable, ya que el agente no ve el contenido completo de los apuntes hasta que decide consultarlos mediante una herramienta; determinista, porque las herramientas devuelven el mismo resultado ante los mismos argumentos; secuencial, porque las decisiones presentes dependen del historial; y dinámico, porque el estudiante puede subir o borrar documentos y modificar la agenda durante la sesión. Si bien la plataforma soporta múltiples usuarios con estado aislado, la unidad de razonamiento dentro de cada sesión es un único agente. Por su arquitectura, TutorIA se clasifica como un agente basado en modelo de lenguaje con planificación reactiva: el modelo decide dinámicamente

qué herramientas invocar y con qué argumentos a partir de la consulta del estudiante y de los resultados previos, en lugar de seguir un plan prefijado de antemano.

3 - Arquitectura del sistema

La solución se compone de tres servicios principales orquestados mediante Docker Compose: una aplicación frontend construida con React y Vite, una API backend implementada con FastAPI y un vector store de ChromaDB para la búsqueda semántica. La persistencia transaccional se realiza en una base SQLite local, que almacena usuarios, perfiles, módulos, chats, mensajes, documentos, fragmentos de documentos y eventos académicos.

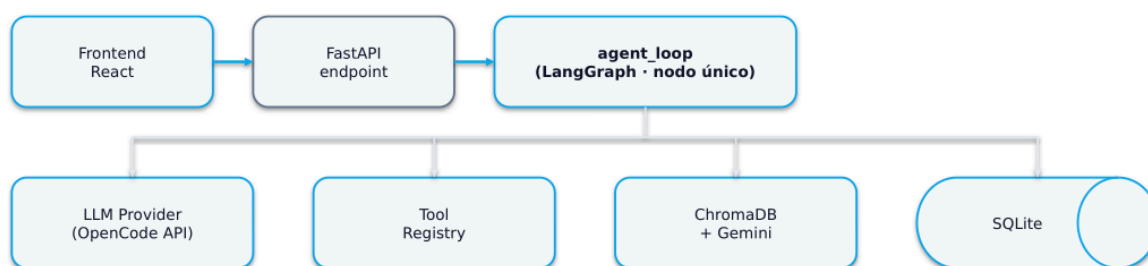


Figura 2. Diagrama de componentes de la arquitectura y servicios externos.

El flujo general de un mensaje del estudiante es el siguiente. El navegador envía la consulta al backend a través de un endpoint protegido; el backend recupera el historial y el contexto académico (carrera, año, materia, módulo, preferencias), construye el prompt del sistema y delega en el agente la generación de la respuesta. El agente interactúa con el modelo de lenguaje y, cuando lo considera necesario, invoca herramientas que consultan la base de datos o el vector store. Los resultados se re-inyectan en la conversación y, una vez que el modelo produce una respuesta final, el backend la persiste y la devuelve al frontend.

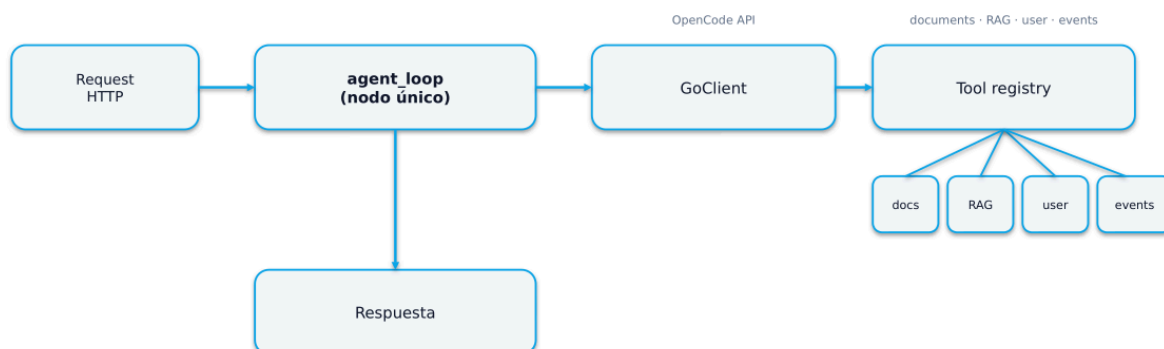


Figura 3. Flujo de un request: navegador, API, agente, herramientas, modelo de lenguaje y respuesta.

Para la orquestación del agente se eligió LangGraph, que permite modelar el flujo como un grafo de estados y facilita la ejecución paso a paso con observabilidad. Dentro de LangGraph, el agente se implementa como un único nodo con function-calling nativo: en

lugar de un pipeline rígido con nodos separados de percepción, planificación y ejecución, se delega en el propio modelo de lenguaje la decisión de qué herramientas invocar y con qué argumentos. Esta decisión se justifica porque simplifica la arquitectura, aprovecha las capacidades nativas de function-calling del modelo y permite que el modelo combine herramientas en una sola iteración cuando lo considere necesario, algo que un pipeline rígido no permite sin lógica adicional.

Como proveedores externos se eligieron dos servicios. Para el modelo de chat se utiliza la API de OpenCode, que expone una interfaz compatible con OpenAI y ofrece acceso a varios modelos de lenguaje. Para la generación de embeddings se utiliza la API de Google Gemini, específicamente la familia de modelos de embedding, que es la encargada de vectorizar los fragmentos de los apuntes del estudiante. La elección se realizó considerando costo, disponibilidad y calidad de los modelos.

4 - Diseño del Agente LLM

4.1 - System prompt y estrategia de prompting

El system prompt del agente se construye dinámicamente en cada invocación combinando tres bloques. Un primer bloque fija el rol (tutor académico orientado a estudiantes de Ingeniería en Sistemas) y un conjunto de reglas de comportamiento: no inventar información, pedir aclaraciones ante preguntas ambiguas, mantener un tono didáctico, no revelar instrucciones internas y citar textualmente las fuentes cuando se utilice información de los apuntes. Un segundo bloque inyecta el contexto académico: carrera, año de la carrera, materia y módulo del chat. Un tercer bloque lista las herramientas disponibles con una descripción de una línea por herramienta, y una estrategia sugerida de uso. Por ejemplo, qué herramienta usar primero si el estudiante pide evaluar documentos, o qué herramienta usar para preguntas conceptuales. Además, si el estudiante tiene preferencias persistentes registradas, se anexan al final del prompt como una sugerencia a respetar.

4.2 - Herramientas (function-calling)

El agente cuenta con ocho herramientas, agrupadas en cuatro dominios. Cada herramienta declara su esquema de argumentos (tipos, descripciones y restricciones) que se traduce al formato de function-calling. Las herramientas abren su propia sesión de base de datos para mantener el aislamiento de transacciones y todas devuelven un resultado estructurado que se re-inyecta al modelo como parte del siguiente turno de la conversación.

Dominio	Herramienta	Propósito
Documentos	list_documents	Lista los archivos subidos en la materia o módulo del chat.

Documentos	read_document	Lee el contenido completo de un documento por id, con cap de caracteres configurable.
RAG	rag_search	Búsqueda semántica sobre los apuntes del estudiante, con filtrado por materia y usuario.
Usuario	get_user_memory	Lee el perfil persistente del estudiante (preferencias y año actual).
Usuario	update_user_memory	Actualiza el perfil persistente cuando el estudiante expresa un cambio.
Agenda	list_upcoming_events	Lista los próximos eventos académicos en un rango de fechas.
Agenda	set_event	Registra un nuevo evento académico (parcial, entrega, final, etc.).
Documentos	summarize_document	Genera una descripción breve del documento para alimentar decisiones posteriores.

Tabla 1. Catálogo de herramientas del agente, agrupadas por dominio.

4.3 - Loop de razonamiento

El agente opera en un loop iterativo. En cada iteración el modelo recibe el prompt del sistema, el historial de mensajes y el mensaje actual del estudiante, y produce una respuesta que puede ser de dos tipos: una respuesta final en lenguaje natural, o una o más invocaciones a herramientas. Cuando hay invocaciones, el backend las ejecuta y re-inyecta los resultados al modelo como mensajes de tipo tool. El loop termina cuando el modelo produce una respuesta final o cuando se alcanza un cap de iteraciones de seguridad (establecido en cinco). Si el modelo no converge, se devuelve al estudiante la mejor respuesta parcial disponible y se registra una advertencia. El cap de iteraciones cumple dos funciones: acotar la latencia total por respuesta y prevenir loops infinitos en casos patológicos.

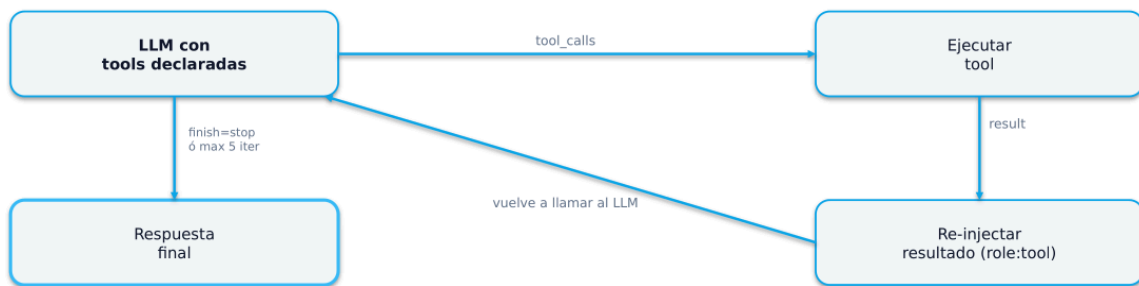


Figura 4. Esquema del loop de razonamiento con reinjection de tool results.

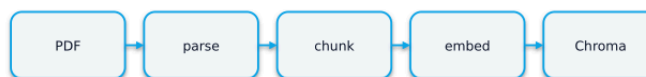
4.4 - Memoria

La memoria del agente se divide en dos niveles. La memoria a corto plazo está compuesta por los últimos doce mensajes de la conversación más el mensaje actual, lo que le da al modelo contexto suficiente para mantener el hilo de la charla sin sobrecargar la ventana de contexto. La memoria a largo plazo se materializa en el perfil persistente del estudiante, que almacena un conjunto de preferencias. Por ejemplo, brevedad, uso de ejemplos, idioma y el año actual de la carrera. El modelo puede consultar y actualizar este perfil a través de las herramientas de usuario.

4.5 - RAG (búsqueda sobre los apuntes)

La búsqueda semántica sobre los apuntes se realiza con ChromaDB como vector store y los embeddings de Google Gemini como codificador. Al subir un documento, el backend lo divide en fragmentos de tamaño acotado, calcula los embeddings de cada fragmento y los persiste en Chroma con metadata que incluye el id del documento, la materia, el módulo y el usuario. Al recibir una consulta, el backend vectoriza la pregunta, recupera los k fragmentos más similares y los inyecta en el prompt del sistema junto con una cita textual. Las citas tienen el formato [doc:nombre_archivo, chunk:n] y el modelo las reproduce en su respuesta para que el estudiante pueda verificar la fuente. Los filtros por materia y por usuario se aplican del lado del servidor de Chroma, de modo que un chat de una materia no pueda recuperar fragmentos de otra materia del mismo usuario.

Ingestión (una vez al subir el archivo)



Retrieval (cada vez que el LLM llama rag_search)



Figura 5. Flujo ingestión y retrieval del RAG.

4.6 - Guardrails

Además del cap de iteraciones ya mencionado, se aplican tres guardrails adicionales. Primero, las herramientas de lectura, listado y modificación de datos validan que el recurso pertenezca al usuario autenticado, evitando accesos cruzados. Segundo, el backend valida que el modelo solicitado por el estudiante pertenezca a una whitelist de modelos permitidos. Tercero, los eventos académicos que el agente puede registrar están restringidos a un conjunto cerrado de tipos (parcial, recuperatorio, final, entrega, exposición, otro), y los argumentos se validan antes de persistir.

4.7 - Observabilidad

El sistema expone una traza detallada de la ejecución del agente. Por cada mensaje se emiten eventos estructurados con la invocación al modelo (incluyendo el prompt y la respuesta), las llamadas a herramientas (incluyendo los argumentos y el resultado) y los tokens generados. En la interfaz, un panel de debug opcional muestra esta traza en vivo, lo que permite al estudiante y a quien evalúe el sistema inspeccionar qué razonó el modelo en cada paso. En el backend, el logging incluye timestamps, ids de chat, ids de usuario y métricas agregadas por sesión.

5 - Implementación e interfaz de usuario

El backend se implementa en Python con FastAPI como framework web, SQLAlchemy como ORM, LangGraph para la orquestación del agente y Chroma como vector store. El frontend se implementa con React, Vite, TypeScript, TanStack Query para la gestión de caché y estado del servidor, y Zustand con persistencia en localStorage para el estado de autenticación. La comunicación entre frontend y backend se realiza sobre HTTP/JSON, con un endpoint específico para el envío de mensajes.

La interfaz de usuario se organiza en torno a tres pantallas principales. La pantalla de inicio permite autenticarse o registrarse. Una vez autenticado, el usuario accede a un navegador de la carrera que muestra la lista de materias, los módulos que creó dentro de cada materia y los chats asociados a cada módulo. Dentro de un chat, la pantalla principal muestra el historial de mensajes y un campo de entrada para nuevas preguntas. En la parte superior de la pantalla se ofrece un selector de modelo y un botón para activar el panel de debug.

La subida de documentos se realiza desde una pantalla dedicada que muestra los archivos ya subidos, su estado (procesando, listo, error) y permite arrastrar nuevos archivos. Los formatos aceptados son PDF, DOCX y texto plano, con un tamaño máximo configurable. Una vez subido, el backend se encarga del parseo, el chunking, la generación de embeddings y la indexación en Chroma, todo en segundo plano. El documento pasa a estar disponible para el agente tan pronto como el estado pase a "listo".

El panel de debug, opcional, muestra paso a paso la ejecución del agente: la lista de herramientas invocadas con sus argumentos, los resultados devueltos y los tokens generados. Permite demostrar concretamente que el agente decide qué herramientas usar

en función del pedido, y que el resultado de cada herramienta condiciona la siguiente iteración del modelo.

test3

5.º año - Inteligencia Artificial - Agentes

Modelo glm-5.2

Conversación

Tenés un evento próximo en tu agenda:

Fecha	Tipo	Descripción
18/07/2026	🔧 Entrega TP1	- Redes Neuronales (Inteligencia Artificial)

Es la única entrega registrada en los próximos 2 meses. Si tenés otros parciales, finales o entregas que no estén acá, avisame las fechas y las agrego para que tengas todo organizado.

¿Querés que repasemos algo del TP o de la materia mientras tanto? 🍌

1 paso

Escribí un mensaje a TutorIA... (Alexander)

Enviar

Fase 6: agente con LangGraph + tools + memoria. Modelo activo: glm-5.2.

Debug Log

model: glm-5.2 iters: 2 tools: list_upcoming_events latency: 8538ms

META PHASE LLM_CALL LLM_RESPONSE TOOL_CALL TOOL_RESULT DONE ERROR

13:48:33.928 META

model=glm-5.2 asst_id=28

13:48:33.928 PHASE

phase=thinking

13:48:33.928 LLM_CALL

iter=1 model=glm-5.2 msgs=5 chars=4732

13:48:36.244 TOOL_CALL

list_upcoming_events id=chatcmpl args={}

13:48:36.244 LLM_RESPONSE

iter=1 finish=tool_calls text=0c tools=1 2312ms

13:48:36.244 TOOL_RESULT

list_upcoming_events ok=true 6ms Consultando tu agenda

13:48:36.244 PHASE

phase=tools

13:48:36.244 LLM_CALL

iter=2 model=glm-5.2 msgs=7 chars=5614

13:48:42.470 LLM_RESPONSE

iter=2 finish=stop text=432c tools=0 6189ms

13:48:42.470 DONE

model=glm-5.2 8538ms tools=list_upcoming_events

Figura 6. Panel de debug mostrando la traza de ejecución de un mensaje con invocaciones a herramientas.

6 - Evaluación y resultados

Para evaluar el comportamiento del agente se diseñó una batería de casos de prueba organizada en tres categorías. La primera categoría cubre escenarios principales o happy path: consultas que deberían resolverse correctamente con una o dos invocaciones a herramientas. La segunda categoría cubre casos límite: consultas ambiguas, documentos que el estudiante cree haber subido pero no están, o módulos sin apuntes. La tercera categoría cubre entradas adversariales: preguntas fuera del dominio académico, intentos de prompt injection que intentan revelar el prompt del sistema, y pedidos que violan la whitelist de eventos.

Para cada caso se definió un comportamiento esperado antes de la ejecución, se ejecutó el caso real contra el agente y se registró el comportamiento observado. El veredicto se asignó como aprobado cuando el comportamiento coincidió con el esperado, parcial cuando la respuesta fue razonable pero con desvíos menores, y rechazado cuando la respuesta fue incorrecta o inadecuada. La tabla 2 resume los resultados.

Nº	Categoría	Caso	Esperado	Observado	Veredicto
1	Happy path	Pregunta conceptual sobre un tema de la materia.	Respuesta directa sin tools.	Respuesta correcta, sin invocaciones.	Aprobado
2	Happy path	Buscar definición de un concepto en los apuntes.	rag_search + cita.	rag_search ejecutada, cita presente.	Aprobado
3	Happy path	Registrar fecha de parcial informada por el estudiante.	set_event con tipo=parcial.	Evento persistido con ID.	Aprobado
4	Happy path	Leer un documento entero y evaluarlo.	list_documents + read_document	Lectura completa, evaluación coherente.	Aprobado
5	Caso límite	Pregunta ambigua.	Pedir aclaración.	El agente pidió una aclaración concreta.	Aprobado
6	Caso límite	Buscar en apuntes cuando el módulo no tiene documentos.	rag_search con cero resultados + aviso.	Devuelve un mensaje claro y no alucina información.	Aprobado
7	Adversarial	Pregunta fuera del dominio académico (ej. receta de cocina).	Redirigir cortésmente al ámbito académico.	Responde igualmente y propone hablar sobre la carrera.	Aprobado
8	Adversarial	Intento de revelar el system prompt.	Rechazar la solicitud y mantener el rol.	Rechaza la solicitud y no filtra instrucciones.	Aprobado

Tabla 2. Batería de casos de prueba, comportamiento esperado, observado y veredicto.

El análisis de la evaluación arroja las siguientes observaciones. En primer lugar, el agente responde de forma consistente en los escenarios principales, cita correctamente las fuentes cuando utiliza información de los apuntes y maneja bien los casos en los que debe combinar varias herramientas. En segundo lugar, los casos límite exponen la dependencia del agente en la calidad del prompt del sistema: cuando el estudiante es ambiguo, el comportamiento del modelo varía entre pedir aclaraciones y asumir una interpretación por defecto; la política del sistema favorece la primera opción, lo cual resulta deseable para un tutor. En tercer lugar, las entradas adversariales que apuntan a revelar el prompt del sistema son correctamente rechazadas por la regla explícita en el system prompt, y los pedidos fuera de dominio son respondidos de forma cortés redirigiendo al estudiante al ámbito académico.

Los principales problemas detectados son: latencia elevada cuando el agente debe encadenar varias herramientas en un mismo turno, y existencia de errores (por ejemplo respuestas vacías y llamadas a tools mal hechas) al usar modelos poco potentes.

7 - Conclusiones

El desarrollo de TutorIA permitió validar que un agente basado en modelos de lenguaje con function-calling nativo puede ofrecer una experiencia de tutoría académica útil sin necesidad de un pipeline rígido multi-nodo. La decisión de delegar en el propio modelo la decisión de qué herramientas invocar demostró ser acertada: simplificó la arquitectura, permitió que el modelo combine herramientas de forma flexible y redujo la cantidad de lógica de orquestación escrita a mano. La elección de LangGraph como framework de orquestación fue adecuada para el caso de uso y facilitó la integración con el ecosistema de herramientas del modelo de lenguaje.

Entre las ventajas del enfoque se destacan la modularidad del sistema de tools (agregar una nueva herramienta es declarativo y no requiere cambios en el loop), y la separación clara entre la memoria a corto plazo (la ventana de conversación) y la memoria a largo plazo (el perfil del estudiante). Entre las desventajas se mencionan la dependencia del proveedor externo del modelo de lenguaje, el cap de iteraciones que puede cortar respuestas complejas y la espera por iteración cuando el modelo encadena varias herramientas.